

Spotify: Content-Based Early Song Promotion Prioritization

GR5243 Applied Data Science — Project II

Team 10 - Boyang Yu, Hantian Zhang, Yingfu Xu, Zixuan Zhu

Department of Statistics, Columbia University

May 2026

1 Business Problem Discussion

Music platforms and record labels often face a resource-allocation problem: among a large number of newly released tracks, only a small subset can receive early recommendation, playlist, or promotional support. However, at the time these decisions are made, strong market feedback such as streams, saves, and listener growth may not yet be available. This creates a need for an early screening framework that can identify high-potential songs using information available before large-scale audience response. We frame this project as a **content-based early song promotion prioritization** problem. The goal is not to fully explain why a song becomes commercially successful, since popularity is affected by many external factors such as artist fanbase, marketing budget, playlist exposure, social-media trends, and release timing. Instead, our goal is more practical: to test whether song-level features can help rank tracks for early promotion more effectively than a simple baseline. This business framing changes the focus of the project from pure prediction accuracy to promotion usefulness. In practice, a platform may only be able to strongly promote the top 10% of candidate songs. Therefore, a useful model should not only predict popularity on average, but also place more true high-potential songs near the top of the ranked list. The central business question of this report is:

Can song-level content, title, and artist-audience signals help prioritize tracks for early promotion before strong market feedback is available?

By answering this question, our project connects statistical modeling with a realistic business decision: how to allocate limited early promotional resources more efficiently.

2 Data Overview

The project uses the same Spotify tracks dataset as Project I. The dataset contains track-level information for approximately 114,000 songs. Each row represents one song, and the variables include song identifiers, artist and album information, Spotify popularity, audio features, and genre labels. Keeping the same dataset as Project I allows us to preserve the original baseline and make a fair comparison between the baseline model and the Project II extensions.

The main outcome variable is popularity, a numerical score ranging from 0 to 100. In the regression task, this score is used directly as the target variable. In the classification task, we convert popularity into a binary hit label. A track is labeled as a hit if its popularity is greater than or equal to the

75th percentile of the training-set popularity distribution. The threshold is computed using the training set only, which avoids using information from the test set when defining the target.

The original Spotify dataset contains several groups of variables. The first group consists of identifiers and text fields, such as `track_id`, `artists`, `album_name`, and `track_name`. These fields are useful for matching and feature construction, but they are excluded from the baseline design matrix as raw identifiers or text. The second group consists of audio features, including `danceability`, `energy`, `loudness`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `valence`, and `tempo`. The third group contains metadata such as `duration_ms`, `explicit`, `key`, `mode`, `time_signature`, and `track_genre`.

Before modeling, we performed basic data cleaning. We removed the unnecessary index column `Unnamed: 0` when present, dropped rows with missing key text identifiers such as artist name, album name, or track name, and reset the row index. After cleaning, the dataset contains 113,999 rows and 20 original columns. We then used a 75% training and 25% test split with a fixed random seed of 42. This fixed split is used consistently throughout the report so that baseline, content-enhanced, Last.fm-enhanced, artist-prior, Random Forest, and XGBoost models are evaluated on the same test set.

For the baseline design matrix, we exclude identifiers, raw text fields, and target variables. The `explicit` variable is converted into a binary 0/1 variable, and `track_genre` is one-hot encoded. Linear models use standardized features fit on the training set only, while tree-based models use the raw tabular feature values. This setup matches the Project I modeling flow and serves as the reference baseline for later comparisons.

Project II then expands the dataset with additional feature groups. Engineered content features capture collaboration structure, duration transformations, audio interactions, and genre-relative deviations. Title-level NLP features summarize basic information from the song title, such as sentiment, length, and keyword indicators. Last.fm features provide external artist-level listener and playcount information. PCA components and KMeans cluster labels summarize audio-feature structure. Finally, internal artist-prior features are computed in a leakage-safe way and used only as a supplementary benchmark.

Table 1. Overview of Main Data Components

Component	Examples	Role in Project
Target variable	popularity	Regression target and source of hit label
Audio features	danceability, energy, loudness, valence, tempo	Core content-based predictors
Metadata	duration_ms, explicit, track_genre	Baseline and engineered predictors
Text fields	artists, album_name, track_name	Used for matching and NLP feature construction
External signals	Last.fm listeners, playcount, tags	Additional artist-audience information
Engineered features	Interactions, genre-relative features, PCA, clusters	Project II feature expansion

Overall, the dataset is well suited for a content-based early screening project because it contains interpretable song-level audio and metadata features. However, it is not sufficient to fully explain commercial success because it lacks direct information on marketing, playlist placement, recommendation exposure, and time-varying listener behavior. This limitation motivates our careful framing: the goal is not full causal explanation, but practical early promotion prioritization.

3 Data Exploration

This section presents the main data exploration and feature engineering steps used in our project. Since our goal is to prioritize songs for early promotion, we focus on features that are available at the track level before strong market feedback is observed. We first examine the target variable and the original Spotify features, then construct additional content-based, title-based, external artist, PCA, and clustering features.

Popularity Distribution and Class Imbalance

The main target variable in the dataset is Spotify popularity, which is a numerical score from 0 to 100. For the regression task, we directly predict this continuous popularity score. For the classification task, we convert popularity into a binary hit label. Following the baseline setup, a song is defined as a hit if its popularity is greater than or equal to the 75th percentile of the training-set popularity distribution.

$$\text{is_hit}_i = \begin{cases} 1, & \text{if } \text{popularity}_i \geq Q_{0.75}(\text{training popularity}), \\ 0, & \text{otherwise.} \end{cases}$$

This definition creates a class imbalance because only the top quarter of songs are labeled as hits. Therefore, accuracy alone is not enough to evaluate classification performance. A model could obtain high accuracy by mostly predicting the majority non-hit class. For this reason, later evaluation also uses precision, recall, F1 score, ROC-AUC, and top-decile lift.

Correlation and Genre Exploration

We first explored the relationships among the original Spotify audio features, including danceability, energy, loudness, acousticalness, instrumentalness, liveness, valence,

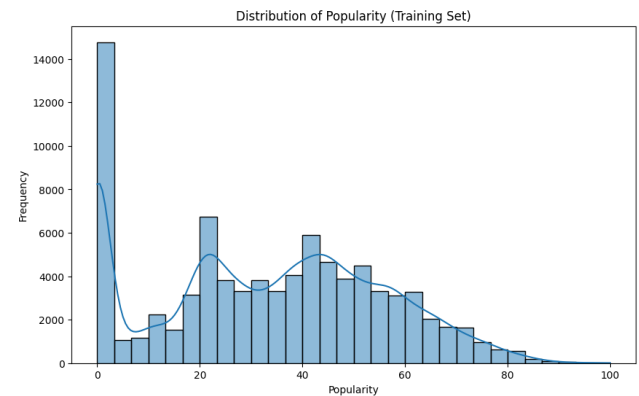


Figure 1. Distribution of Spotify popularity scores in the training set. The binary hit label is defined using the 75th percentile of the training-set popularity distribution.

and tempo. The correlation heatmap helps us understand whether some audio variables move together and whether simple linear relationships are likely to explain song popularity.

The exploration shows that audio features contain useful information, but no single audio feature can clearly separate hits from non-hits. This motivates the use of interaction features and nonlinear models. We also explored genre-level patterns because a song's audio characteristics should be interpreted relative to its genre. For example, high energy may be common in some genres but unusual in others. Therefore, we later create genre-relative features to compare a song against the average song in its own genre.

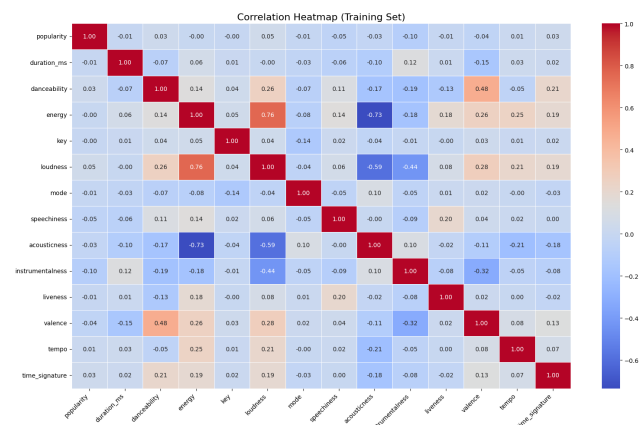


Figure 2. Correlation heatmap of core Spotify audio features.

Original Spotify Features

The original dataset contains track-level audio features and metadata features. The main audio features include:

- **danceability**: how suitable a track is for dancing;
- **energy**: the perceived intensity and activity of the track;
- **loudness**: the overall loudness of the track;
- **acousticalness**: whether the track is likely to be acoustic;

- **instrumentalness**: whether the track is likely to contain vocals;
- **liveness**: whether the track sounds like a live performance;
- **valence**: the musical positiveness of the track;
- **tempo**: the estimated beats per minute.

The metadata features include duration, explicit-content indicator, genre, artist information, album name, and track name. The baseline model uses the original tabular variables and one-hot encoded genre indicators. However, the original variables have limited semantic information and limited feature interactions, so we construct additional engineered features for Project II.

Engineered Content Features

We created several groups of engineered content features to capture collaboration structure, duration effects, audio interactions, and genre-relative signals.

First, we extracted collaboration-related features from the `artists` field. The feature `num_artists` counts the number of artists on a track, and `is_collab` indicates whether a song has more than one artist:

$$\text{is_collab}_i = \begin{cases} 1, & \text{if song } i \text{ has more than one artist,} \\ 0, & \text{otherwise.} \end{cases}$$

This is useful because collaborations may receive exposure from multiple artists' audiences.

Second, we transformed the duration variable. The raw duration is measured in milliseconds, so we converted it into minutes:

$$\text{duration_min}_i = \frac{\text{duration_ms}_i}{60000}.$$

We also created a log-transformed duration feature:

$$\text{duration_log}_i = \log(1 + \text{duration_ms}_i).$$

The log transformation reduces the effect of extremely long songs. In addition, we created duration buckets using training-set tertiles, which allows the model to capture nonlinear differences between short, medium, and long tracks.

Third, we created audio interaction features. These features allow the model to capture combined musical characteristics instead of treating each audio variable independently:

$$\begin{aligned} \text{energy_loudness}_i &= \text{energy}_i \times \text{loudness}_i, \\ \text{danceability_energy}_i &= \text{danceability}_i \times \text{energy}_i, \\ \text{valence_energy}_i &= \text{valence}_i \times \text{energy}_i. \end{aligned}$$

For example, a song that is both danceable and energetic may have a different promotion potential than a song that is only high in one of these dimensions.

Finally, we created genre-relative features for energy, danceability, and valence. For each song, we subtract the training-set genre mean:

$$x_{\text{minus_genre_mean},i} = x_i - \bar{x}_{g(i),\text{train}},$$

where x is one of the audio features and $g(i)$ is the genre of song i . Using training-set genre means avoids test-set leakage. These features measure whether a song is above or below the typical level of its own genre.

Table 2. Main engineered content features.

Feature Group	Features	Purpose
Collaboration	<code>num_artists</code>	Multi-artist exposure
	<code>is_collab</code>	
Duration	<code>duration_min</code>	Song length effects
	<code>duration_log</code>	
	duration buckets	
Audio interactions	<code>energy_loudness</code>	Combined audio patterns
	<code>danceability_energy</code>	
	<code>valence_energy</code>	
Genre-relative	Audio feature minus genre mean	Within-genre comparison

Title-Level NLP Features

In addition to numerical audio variables, we extracted simple NLP features from `track_name`. These features are designed to capture weak title-level signals that may be useful for early promotion prediction.

First, we used VADER sentiment analysis to compute a title sentiment score. This feature measures the emotional tone of the song title. We also measured title complexity using word count and character length:

`title_word_counti` = number of words in the title,

`title_char_leni` = number of characters in the title.

We also created several binary title indicators:

- **title_has_feat**: whether the title contains terms such as “feat”, “ft”, “featuring”, or “with”;
- **title_has_remix**: whether the title contains terms such as “remix”, “mix”, “edit”, or “version”;
- **title_is_short**: whether the title has two or fewer words;
- **title_has_love**: whether the title contains the word “love”;
- **title_has_number**: whether the title contains a number;

- `title_exclamation`: whether the title contains an exclamation mark.

These features are simple and interpretable. They are also available before the song receives strong market feedback, so they are appropriate for an early song promotion setting.

External Artist Signals from Last.fm

We also merged cached Last.fm artist-level information into the Spotify dataset. Last.fm is a music tracking and recommendation platform that records users' listening histories. Its artist pages provide audience-related statistics such as listener count, playcount, and tags.

For each Spotify track, we extracted the primary artist from the `artists` field and used it as the merge key. Then we added the following Last.fm features:

- `lfm_matched`: whether the artist was matched in the Last.fm cache;
- `lfm_listeners_log`: log-transformed listener count;
- `lfm_playcount_log`: log-transformed playcount;
- `lfm_play_per_listener_log`: log-transformed playcount per listener.

The log transformations reduce the effect of extremely popular artists:

$$\begin{aligned} \text{lfm_listeners_log} &= \log(1 + \text{lfm_listeners}), \\ \text{lfm_playcount_log} &= \log(1 + \text{lfm_playcount}). \end{aligned}$$

These Last.fm features provide external artist-level popularity signals. However, they are not pure audio-content features. Therefore, in the modeling section, we treat Content + NLP as the pure content upgrade and Content + NLP + Last.fm as the enriched model with external artist information.

PCA for Audio Feature Representation

We applied Principal Component Analysis to the standardized continuous audio features. The scaler and PCA model were fit on the training set only and then applied to the test set. This prevents information from the test set from entering the feature construction process.

The first principal component explains 31.97% of the variance in the standardized audio features, and the second principal component explains 15.80%. Together, PC1 and PC2 explain 47.78% of the total audio-feature variance. We added PC1 and PC2 as compact audio representation features. The PCA scatter plot shows that hit and non-hit songs overlap heavily in the PC1–PC2 space. This means that audio features contain some useful structure, but the first two principal components alone cannot clearly separate successful songs from unsuccessful songs. Therefore, PCA is mainly used for representation and visualization rather than as direct evidence that audio features fully explain popularity.

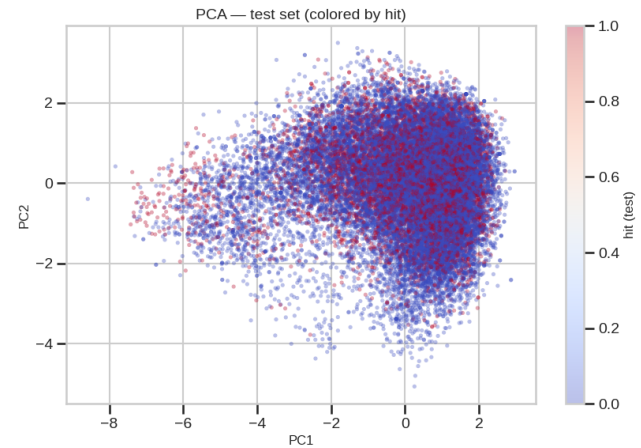


Figure 3. PCA visualization of the test set colored by hit status. Hit and non-hit songs overlap heavily in the first two principal components.

KMeans Clustering for Song Profiles

We used KMeans clustering on the standardized audio features to create interpretable song profiles. The purpose of clustering is not to directly predict popularity, but to summarize songs into broad content-based groups and support business interpretation.

We tested $k = 2$ to $k = 10$ using inertia and silhouette score. The silhouette-best value was $k = 2$, but we selected $k = 3$ for better interpretability. With only two clusters, the groups may be too broad to describe useful song profiles. Using three clusters gives a more practical segmentation while keeping the structure simple.

The final cluster labels were added as new model features. These labels allow the model to use broad audio-profile information in addition to individual audio variables.

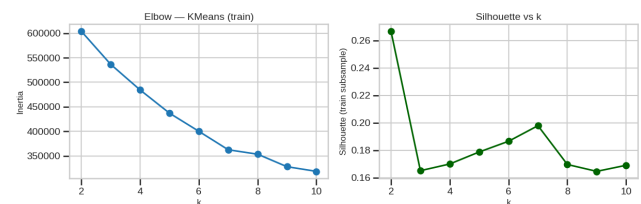


Figure 4. KMeans clustering diagnostics using inertia and silhouette score for $k = 2, \dots, 10$.

Overall, the data exploration and feature engineering process expands the original Spotify dataset into a richer set of predictors. These include original audio features, engineered content signals, title-level NLP features, external Last.fm artist signals, PCA components, and cluster labels. These features support the main project goal: ranking songs for early promotion before strong market feedback becomes available.

4 Methodology

To evaluate whether Spotify song characteristics can be used to predict song success, we designed the modeling process around two related prediction tasks. The first task was a regression problem, where the goal was to predict the continuous Spotify popularity score. The second task was a classification problem, where the goal was to predict whether a track would be classified as a hit. A hit was defined based on the training-set popularity threshold, and the same train-test split and target definition were kept consistent across model families so that the performance comparisons would be fair.

For evaluation, we used different metrics for the two tasks. For the regression models, we reported root mean squared error (RMSE), mean absolute error (MAE), and R^2 . RMSE and MAE measure the size of prediction errors, while R^2 shows how much variation in popularity can be explained by the model. For the classification models, we reported accuracy, precision, recall, F1 score, and ROC-AUC. Because hit songs are relatively rare compared with non-hit songs, accuracy alone is not sufficient. Precision, recall, F1, and ROC-AUC provide a more complete view of whether the model can identify high-potential tracks rather than simply predicting the majority class.

We used a controlled feature ladder to understand how each additional source of information improved model performance. The baseline model used the original Spotify variables and genre dummies. The second model family, Content + NLP, added engineered content features, PCA components, cluster labels, and title-level NLP features. The third model family, Content + NLP + Last.fm, further added four external artist-level signals from Last.fm. Finally, the artist-prior supplement added internal train-safe artist-prior features and was treated as a supplementary upper-bound model rather than the main deployable model. This structure allowed us to separate the value of pure content features, external artist popularity signals, and internal artist history.

The main modeling backbone was Random Forest. This choice was based on the baseline results, where Random Forest performed better than simpler linear models and single decision trees for both regression and classification tasks. Random Forest is well suited for this problem because it can capture nonlinear relationships and interactions among audio, genre, title, and artist-level features without requiring a strict linear functional form.

We also trained XGBoost models as a robustness comparison. The XGBoost models used 200 estimators, a maximum depth of 5, a learning rate of 0.05, a subsample rate of 0.9, and a column subsample rate of 0.9. For the classification task, `scale_pos_weight` was used to address class imbalance.

5 Model Results

Random Forest Results

Random Forest was the strongest and most stable model family in the main analysis. In the regression task, the baseline Ran-

dom Forest achieved an RMSE of 15.4366, MAE of 10.4530, and R^2 of 0.5201. After adding content engineering and title NLP features, performance improved slightly, with RMSE decreasing to 15.0769 and R^2 increasing to 0.5422. This suggests that engineered content and title-level features add useful information, although the improvement from content-only upgrades was relatively modest.

The largest improvement came after adding Last.fm external artist signals. The Content + NLP + Last.fm Random Forest model achieved the best main regression performance, with RMSE decreasing to 13.0343, MAE decreasing to 8.4227, and R^2 increasing to 0.6578. Compared with the baseline model, this is a substantial improvement in both prediction error and explanatory power. The artist-prior supplement also improved over the baseline, with RMSE of 14.6741, MAE of 8.7463, and R^2 of 0.5663, but it did not outperform the Last.fm model in regression.

Table 3. Random Forest Regression Results

Feature Family	RMSE	MAE	R^2
Baseline	15.4366	10.4530	0.5201
Content + NLP	15.0769	10.4250	0.5422
Content + NLP + Last.fm	13.0343	8.4227	0.6578
Artist-prior supplement	14.6741	8.7463	0.5663

The classification results show a similar pattern. The baseline Random Forest reached an accuracy of 0.8440, precision of 0.7909, recall of 0.5303, F1 score of 0.6349, and ROC-AUC of 0.8956. Adding Content + NLP features slightly improved accuracy and ROC-AUC, but recall decreased from 0.5303 to 0.5108. This indicates that the content-only model became more conservative in identifying hits, even though its overall ranking ability improved slightly.

The Content + NLP + Last.fm model produced a much stronger classification result. Accuracy increased to 0.8816, precision reached 0.8682, recall improved to 0.6332, F1 increased to 0.7323, and ROC-AUC rose to 0.9354. This indicates that Last.fm artist-level audience signals provide important predictive information beyond audio and title features. The artist-prior supplement achieved the highest classification performance overall, with accuracy of 0.8924, recall of 0.7129, F1 of 0.7722, and ROC-AUC of 0.9434. However, because this model relies on internal artist-prior information, it should be interpreted as a supplementary model rather than the main deployable content-based model.

Table 4. Random Forest Classification Results

Feature Family	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline	0.8440	0.7909	0.5303	0.6349	0.8956
Content + NLP	0.8501	0.8408	0.5108	0.6355	0.9019
Content + NLP + Last.fm	0.8816	0.8682	0.6332	0.7323	0.9354
Artist-prior supplement	0.8924	0.8423	0.7129	0.7722	0.9434

Overall, the Random Forest results suggest that audio and con-

tent signals are useful, but not sufficient on their own. The Content + NLP model produced only a small lift over the baseline. In contrast, adding Last.fm features created a large improvement in both regression and classification. This supports the idea that song success is not determined only by what a song sounds like; artist-level audience exposure and historical listener engagement also provide strong predictive signals.

XGBoost Comparison

We also trained XGBoost models for the three extended feature families: Content + NLP, Content + NLP + Last.fm, and Artist-prior supplement. The purpose was not to replace Random Forest directly, but to test whether a stronger boosting-based tree model would produce similar patterns.

For regression, XGBoost improved as more informative feature groups were added. The Content + NLP XGBoost model had an RMSE of 19.0065 and R^2 of 0.2724, which was weaker than the corresponding Random Forest model. Adding Last.fm improved the XGBoost regression model to RMSE 18.1380 and R^2 0.3374. The artist-prior supplement performed best among the XGBoost regression models, with RMSE 14.2528, MAE 9.0685, and R^2 0.5909.

Table 5. XGBoost Regression Results

Feature Family	RMSE	MAE	R^2
Content + NLP	19.0065	15.0691	0.2724
Content + NLP + Last.fm	18.1380	14.1577	0.3374
Artist-prior supplement	14.2528	9.0685	0.5909

For classification, XGBoost had relatively high recall but lower precision than Random Forest. The Content + NLP XGBoost model achieved recall of 0.7697 but precision of only 0.4543, resulting in an F1 score of 0.5714 and ROC-AUC of 0.8146. After adding Last.fm, recall increased to 0.8303 and ROC-AUC improved to 0.8618, but precision remained relatively low at 0.5086. The artist-prior supplement again produced the strongest XGBoost classification result, with accuracy of 0.8346, precision of 0.6332, recall of 0.8394, F1 of 0.7219, and ROC-AUC of 0.9203.

Table 6. XGBoost Classification Results

Feature Family	Accuracy	Precision	Recall	F1	ROC-AUC
Content + NLP	0.7046	0.4543	0.7697	0.5714	0.8146
Content + NLP + Last.fm	0.7514	0.5086	0.8303	0.6308	0.8618
Artist-prior supplement	0.8346	0.6332	0.8394	0.7219	0.9203

The XGBoost results confirm the general direction of the Random Forest results: Last.fm features improve performance, and artist-prior features are highly informative. However, the untuned XGBoost models did not outperform Random Forest in the main model families. In particular, XGBoost tended to capture more true hits, as shown by its higher recall, but it also produced more false positives, as reflected in its lower precision. Therefore, XGBoost is useful as a robustness check, but

Random Forest remains the better main model for this report because it provides stronger overall performance and a better balance between precision and recall.

Summary of Model Findings

The model results lead to three main findings. First, Random Forest consistently outperformed simpler baseline models and served as the strongest main modeling approach. Second, engineered content and title-level NLP features provided a small but consistent improvement over the baseline, showing that content signals do contain predictive value. Third, Last.fm artist-level features produced the largest main-model improvement, raising Random Forest regression R^2 from 0.5422 to 0.6578 and classification ROC-AUC from 0.9019 to 0.9354 when added to Content + NLP features. Although the artist-prior supplement achieved the best classification metrics overall, it should be treated carefully because it relies on internal artist-history information. For the final business recommendation, the Content + NLP + Last.fm Random Forest model is the strongest main deployable model. It offers a good balance between predictive performance and practical interpretability, while avoiding over-reliance on internal artist-prior signals.

6 Results and Recommendations

6.1 Business Evaluation: Top-Decile Lift

In a real early promotion settings, music platform or record label are usually limited to resources, including marketing budgets, playlists slots, they are not able to promote every new release simultaneously. To maximize the profit, rather than asking “how well does the model predict on average?”, the operationally-related question for these stakeholders is therefore “How many true hits does the model can help us to find if we can only bet on the top 10% of new tracks?”

Here, the statistical metrics are transitioning from RMSE or ROC-AUC which were used for measuring the overall prediction quality to a top decile lift analysis that better satisfy the project demands. To evaluate this, we ranked all the test-songs by each model’s predicted score and filtered with the top 10% of the songs. The test set contain 28,500 songs in total, which 2,850 songs will be selected under the random promotion strategy with no model guidance. There are 7,290 true hits among all test tracks 729 will be randomly selected from 2,850 tracks, resulting to a precision of 25.6%, which was set as the benchmark of a lift of 1.0×.

Table 7. Top-Decile Lift Analysis — Random Forest Classifier

Feature Family	Hit Capture Rate	Precision @ Top 10%	Lift vs. Random
Baseline RF Classifier	32.4%	83.0%	3.25×
Content + NLP RF	34.5%	88.3%	3.45×
Content + NLP + Last.fm RF	36.3%	92.9%	3.64×
Artist-prior RF	37.3%	95.3%	3.73×

From Table 7, we can clearly see a consistent progression pattern across model families. Even the baseline model delivers a substantial growth over changes, covering 32.4% of the true hits at a precision rate of 83%, gaining more than 3 times better

than the random benchmark. The capture rate further increased with the introduction of engineered content features and title-level NLP features, showing the business value of track-level content.

Although Artist-prior supplement does captured the most hits with a lift of $3.73\times$, the model need to be carefully reevaluate for its properties of heavy rely on the internal historical artist-prior information. In other words, as we have discussed in section 4, the model is more suitable as a supplementary upper-bound model rather than a content-based deployable model.

The strongest main deployable model here is the Content + NLP + Last.fm Random Forest, capturing 2,648 hits with 92.9% precision in the promoted group, corresponding with 3.64 times better performance and a robust statistical metrics performance, showing the value of Last.fm artist-audience signals. The model was further validated from a business perspective, it improved the promotion list in a practical way by capturing 133 additional hits, demonstrating an operationally value of deciding which songs deserve the early promotional attention. Excelling according accuracy, ranking quality, interpretability and deployability, the Content + NLP + Last.fm Random Forest stands out as our top recommended model.

6.2 Feature Importance and Interpretation

To further understand the advantages of the Content + NLP + Last.fm Random Forest classifier, we analyzed its feature importance, gaining deeper insight into the types of signals the model relies on for promotion ranking.

Table 8. Feature Group Importance — Content + NLP + Last.fm RF Classifier

Feature Group	Importance Share
Original Spotify Audio Features	24.6%
Engineered Content Features	22.4%
Last.fm External Artist Features	21.0%
Genre / Cluster / Duration Buckets	15.0%
NLP Title Features	6.5%
PCA Audio Components	5.6%
Other	4.9%

The feature-importance breakdown shows that the model does not depend on a single variable group, the top three features sharing similar weights: Original Spotify Audio Features remain the largest group, accounting 24.6%; Engineered Content Features contribute to another 22.4%, showing the meaning information added from engineering step; Last.fm External Artist Features account for 21.0%, explaining why their inclusion produces the largest improvement in predictive performance. `lfm_playcount_log` (8.1%), `lfm_listeners_log` (7.2%), and `lfm_play_per_listener_log` (5.5%) together made up of the Last.fm feature, indicating the importance of artist reputation related signals including play count and listener groups of early promotion prioritization.

The importance decomposition reveals that the popularity of a song is both influenced by both track-level content and artist-

level audience exposure. Audio and engineered content features describe the attributes and properties of song, while Last.fm feature approximate the audience scale and level of listeners' engagement.

Though NLP Title Features only contribute a modest 6.5%, it still should be treated as secondary signals. Given the features are extracted directly from the track name with no need for the audio analysis, it is economically valuable, adding signal with zero marginal cost per song.

6.3 Business Recommendations

The following recommendations translate the modeling result to practical operational guidance for stakeholder like music platform or record label

Integrate top-decile scoring into song release procedure.

The system will be leveraged as a promotion-ranking tool, which the stakeholders apply Content + NLP + Last.fm Random Forest Classifier to rate each new songs during upload. The top decile songs will automatically trigger early playlist considerations and algorithmic boost eligibility, which combine subjective judgment and data-driven filters to find the best promotion strategy and manage resources.

Content based signal prioritization for cold-start artists.

For new artists missing historical results on Last.fm, the model will automatically falls back to the Content + NLP features. Though $3.45\times$ lift is delivered, the potential of these artists may be underestimated. Under the circumstances of missing reputation metric, the company should focus on the quality of audio, the fit of genre and characteristics of title for these new artists' early feedback.

Use genre-relative positioning. The engineered genre features do contribute to the model, while it also indicates differences. A song that is substantially more energetic doesn't mean it's better, but it can potentially differentiate itself from other released music, can be identified as a signal that might worth promoting.

Monitor top-decile precision. The precision is varied over time, and may drift with the evolvement of song catalog and shifts of market trend. The model should be retrained periodically to preserve its discrimination power.

7 Future Work, Limitations

7.1 Limitations

Static snapshot dataset. Lack of time dimension fails to capture a clean picture of the song's behaviors and peak momentum after launch.

External artist popularity signals. Though Last.fm features have significant improvement toward the model performance, they also make the model poorly content-based by introducing the information of artists, which may cause underestimation for niche artists.

Artist-prior leakage risk. As artist-prior supplement leverages the artist-level from same dataset and computed in a train-safe way, it can still only be considered as a performance ceiling rather than a deployable model even if it has the best results.

Oversimplified representation of genres. The current model uses genre dummy variables, cluster labels and bucket features, while genre itself is complex and multi-label. Simple one-hot encoding may easily overfit and hard to capture the relationship between similar genres, ignoring the hierarchy within genre structure.

7.2 Future Work

Time-series modeling. Building a truly forward-looking model by using the older releases to train the model, using newer releases to test the model, providing a more honest estimate of deployment performance.

Early streaming and engagement signals. Incorporate early stream counts, save rates, skip rates, completion rates and listener growth as real-time feedback signals with a time frame of 24 to 72 hours, to directly reflect the early momentum and fit the model better to the practical promotion timing.

Genre representation improvement. Introduce multi-label genre tags, hierarchical genre groups to reduce overfitting and better capture the relationship between similar music.

Ensemble and stacking. The current comparison treats XGBoost and RandomForest as independent models, combining the predictions of multiple base learners could improve robustness, especially for the case that near to hit or non-hit threshold.

Fairness and diversity analysis. Examining whether the model systematically under-scores certain artist and introducing fairness-aware training objectives or reranking if biases are detected.

References

[1] Spotify Tracks Dataset (Kaggle).

<https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>

[2] Spotify Tracks Dataset (Kaggle / GitHub mirror).

https://github.com/Boyang6000/stat_5243_project1

[3] Last.fm API. Artist listener and playcount statistics. <https://www.last.fm/api>

A Appendix

A.1 Notebook Reference

All code is in https://colab.research.google.com/drive/1EoG0vTGzwjQH3_c3wos6G3m8piwepD1?usp=sharing. Selected and condensed key sections:

Listing 1. Target Definition and Train-Test Split

```
def build_baseline_design_matrix(df):
    """Exclude identifiers/text/targets and one-hot categorical features."""
    exclude_cols = [
        "track_id", "artists", "album_name", "track_name",
        "popularity", "is_hit",
        "lfm_listeners", "lfm_playcount", "lfm_top_tag",
        "instrumental_bin"
    ]

    feature_cols = [c for c in df.columns if c not in exclude_cols]
    X = df[feature_cols].copy()

    if "explicit" in X.columns:
        X["explicit"] = X["explicit"].astype(int)

    categorical_cols = []
    if "track_genre" in X.columns:
        categorical_cols.append("track_genre")
    if "cluster" in X.columns:
        X["cluster"] = X["cluster"].astype(int).astype(str)
        categorical_cols.append("cluster")

    if categorical_cols:
        X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

    return X

n = len(data)
train_idx, test_idx = train_test_split(
    np.arange(n),
    test_size=0.25,
    random_state=RANDOM_STATE
)

X_base_full = build_baseline_design_matrix(data)
y_reg = data["popularity"].astype(float)

X_train_base = X_base_full.iloc[train_idx].copy()
X_test_base = X_base_full.iloc[test_idx].copy()
y_train_reg = y_reg.iloc[train_idx].values
y_test_reg = y_reg.iloc[test_idx].values

hit_threshold = float(pd.Series(y_train_reg).quantile(0.75))
y_train_class = (y_train_reg >= hit_threshold).astype(int)
y_test_class = (y_test_reg >= hit_threshold).astype(int)

scaler_base = StandardScaler()
X_train_base_scaled = scaler_base.fit_transform(X_train_base)
X_test_base_scaled = scaler_base.transform(X_test_base)

def reg_metrics(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    return {
        "RMSE": float(np.sqrt(mse)),
        "MAE": float(mean_absolute_error(y_true, y_pred)),
        "R2": float(r2_score(y_true, y_pred)),
    }

def clf_metrics(y_true, y_prob, y_pred):
    out = {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "precision": float(precision_score(y_true, y_pred, zero_division=0)),
        "recall": float(recall_score(y_true, y_pred, zero_division=0)),
        "f1": float(f1_score(y_true, y_pred, zero_division=0)),
    }
    try:
        out["roc_auc"] = float(roc_auc_score(y_true, y_prob))
    except ValueError:
        out["roc_auc"] = float("nan")
    return out
```

Listing 2. Feature Engineering Pipeline

```
# Title-level NLP features
analyzer = SentimentIntensityAnalyzer()

NLP_COLS = [
    "title_sentiment", "title_word_count", "title_char_len",
    "title_has_feat", "title_has_remix", "title_is_short",
    "title_has_love", "title_has_number", "title_exclamation",
]

def extract_title_nlp(name):
    if not isinstance(name, str):
        return {k: 0 for k in NLP_COLS}

    name_lower = name.lower()
    s = analyzer.polarity_scores(name)
    words = re.findall(r"[a-zA-Z]+", name_lower)

    return {
        "title_sentiment": s["compound"],
        "title_word_count": len(words),
        "title_char_len": len(name),
        "title_has_feat": int(bool(re.search(r"\bfeat\b|\bft\b|\bfeaturing\b|\bwith\b|&", name_lower))),
        "title_has_remix": int(bool(re.search(r"\bremix\b|\bmix\b|\bedit\b|\bversion\b", name_lower))),
        "title_is_short": int(len(words) <= 2),
        "title_has_love": int("love" in name_lower),
        "title_has_number": int(bool(re.search(r"d", name))),
        "title_exclamation": int("!" in name),
    }

data = data.drop(columns=[c for c in NLP_COLS if c in data.columns], errors="ignore")
nlp_feats = data["track_name"].apply(extract_title_nlp)
nlp_df = pd.DataFrame(nlp_feats.tolist(), index=data.index)
```



```

data = pd.concat([data, nlp_df], axis=1)

# Cached Last.fm artist-level features. No API key is used here.
def normalize_primary_artist(artist_value):
    if pd.isna(artist_value):
        return np.nan
    primary = str(artist_value).split(";")[0].strip()
    return primary if primary else np.nan

data["primary_artist"] = data["artists"].apply(normalize_primary_artist)

numeric_lfm_cols = [
    "lfm_listeners", "lfm_playcount", "lfm_matched",
    "lfm_listeners_log", "lfm_playcount_log",
    "lfm_play_per_listener", "lfm_play_per_listener_log",
]

for col in ["lfm_listeners", "lfm_playcount"]:
    df_lfm[col] = pd.to_numeric(df_lfm[col], errors="coerce").fillna(0)

df_lfm["lfm_matched"] = (
    (df_lfm["lfm_listeners"] > 0) |
    (df_lfm["lfm_playcount"] > 0)
).astype(int)

df_lfm["lfm_listeners_log"] = np.log1p(df_lfm["lfm_listeners"])
df_lfm["lfm_playcount_log"] = np.log1p(df_lfm["lfm_playcount"])
df_lfm["lfm_play_per_listener"] = (
    df_lfm["lfm_playcount"] / df_lfm["lfm_listeners"].replace(0, np.nan)
)
df_lfm["lfm_play_per_listener_log"] = np.log1p(
    df_lfm["lfm_play_per_listener"]
).replace([np.inf, -np.inf], np.nan)
.fillna(0)

lfm_merge_cols = [
    "primary_artist", "lfm_artist_name", "lfm_listeners",
    "lfm_playcount", "lfm_top_tag", "lfm_api_status",
    "lfm_matched", "lfm_listeners_log", "lfm_playcount_log",
    "lfm_play_per_listener", "lfm_play_per_listener_log",
]

old_lfm_cols = [c for c in lfm_merge_cols if c != "primary_artist" and c in data.columns]
data = data.drop(columns=old_lfm_cols, errors="ignore")
data = data.merge(
    df_lfm[lfm_merge_cols].drop_duplicates(subset=["primary_artist"]),
    on="primary_artist",
    how="left"
)

for col in numeric_lfm_cols:
    data[col] = pd.to_numeric(data[col], errors="coerce").fillna(0)

# Content-based engineered features
df = data.copy()
train_ids = df.index[train_idx]
test_ids = df.index[test_idx]

df["artists"] = df["artists"].astype(str)
artist_parts = df["artists"].str.split(";")

df["num_artists"] = artist_parts.apply(
    lambda x: len([a.strip() for a in x if isinstance(a, str) and a.strip() != ""])
)
df["num_artists"] = df["num_artists"].replace(0, 1)
df["is_collab"] = (df["num_artists"] > 1).astype(int)

df["duration_min"] = df["duration_ms"] / 60000.0
df["duration_log"] = np.log1p(df["duration_ms"].astype(float))

q1, q2 = df.loc[train_ids, "duration_min"].quantile([1 / 3, 2 / 3]).values
df["duration_bucket"] = np.where(
    df["duration_min"] <= q1,
    "short",
    np.where(df["duration_min"] <= q2, "medium", "long")
)

df["energy_loudness"] = df["energy"] * df["loudness"]
df["danceability_energy"] = df["danceability"] * df["energy"]
df["valence_energy"] = df["valence"] * df["energy"]

# Genre-relative features use train-set genre means only.
for c in ["energy", "danceability", "valence"]:
    genre_mean_map = df.loc[train_ids].groupby("track_genre")[c].mean()
    global_feature_mean = df.loc[train_ids, c].mean()
    mean_col = f"{c}.genre_train_mean"
    rel_col = f"{c}.minus_genre_mean"
    df[mean_col] = df["track_genre"].map(genre_mean_map).fillna(global_feature_mean)
    df[rel_col] = df[c] - df[mean_col]

# Artist-prior supplement uses train-safe artist history.
df["hit_tr"] = np.nan
df.loc[train_ids, "hit_tr"] = (
    df.loc[train_ids, "popularity"] >= hit_threshold
).astype(int)

train_artist = df.loc[train_ids, ["primary_artist", "popularity", "hit_tr"]].copy()
artist_stats = train_artist.groupby("primary_artist").agg(
    artist_train_count=("popularity", "size"),
    artist_pop_sum=("popularity", "sum"),
    artist_hit_sum=("hit_tr", "sum")
)

global_avg_pop = float(train_artist["popularity"].mean())
global_hit_rate = float(train_artist["hit_tr"].mean())

df["artist_song_count"] = np.nan
df["artist_avg_popularity"] = np.nan
df["artist_hit_rate"] = np.nan

train_tmp = train_artist.join(artist_stats, on="primary_artist")
loo_count = train_tmp["artist_train_count"] - 1

```

```

loo_pop_sum = train_tmp["artist_pop_sum"] - train_tmp["popularity"]
loo_hit_sum = train_tmp["artist_hit_sum"] - train_tmp["hit_tr"]

loo_avg_pop = pd.Series(global_avg_pop, index=train_tmp.index)
loo_hit_rate = pd.Series(global_hit_rate, index=train_tmp.index)
has_history = loo_count > 0
loo_avg_pop.loc[has_history] = loo_pop_sum.loc[has_history] / loo_count.loc[has_history]
loo_hit_rate.loc[has_history] = loo_hit_sum.loc[has_history] / loo_count.loc[has_history]

df.loc[train_ids, "artist_song_count"] = loo_count.values
df.loc[train_ids, "artist_avg_popularity"] = loo_avg_pop.values
df.loc[train_ids, "artist_hit_rate"] = loo_hit_rate.values

test_artist = df.loc[test_ids, ["primary_artist"]].join(artist_stats, on="primary_artist")
df.loc[test_ids, "artist_song_count"] = test_artist["artist_train_count"].fillna(0).values
df.loc[test_ids, "artist_avg_popularity"] = (
    test_artist["artist_pop_sum"] / test_artist["artist_train_count"]
).replace([np.inf, -np.inf], np.nan).fillna(global_avg_pop).values
df.loc[test_ids, "artist_hit_rate"] = (
    test_artist["artist_hit_sum"] / test_artist["artist_train_count"]
).replace([np.inf, -np.inf], np.nan).fillna(global_hit_rate).values

# PCA and KMeans audio representation features.
audio_cols = [
    "danceability", "energy", "loudness", "speechiness", "acousticness",
    "instrumentalness", "liveness", "valence", "tempo"
]

scaler_audio = StandardScaler()
X_audio_train = scaler_audio.fit_transform(df.loc[train_ids, audio_cols])
X_audio_all = scaler_audio.transform(df[audio_cols])

pca = PCA(n_components=2, random_state=RANDOM_STATE)
pca.fit(X_audio_train)
pc_all = pca.transform(X_audio_all)
df["PC1"] = pc_all[:, 0]
df["PC2"] = pc_all[:, 1]

kmeans = KMeans(n_clusters=3, random_state=RANDOM_STATE, n_init=20)
kmeans.fit(X_audio_train)
df["cluster"] = kmeans.predict(X_audio_all)

```

Listing 3. Random Forest Model Training and Evaluation

```

def clean_numeric_matrix(X):
    X = X.copy()
    bool_cols = X.select_dtypes(include=["bool"]).columns.tolist()
    if bool_cols:
        X[bool_cols] = X[bool_cols].astype(int)
    non_numeric_cols = X.select_dtypes(exclude=[np.number]).columns.tolist()
    if non_numeric_cols:
        X = X.drop(columns=non_numeric_cols, errors="ignore")
    return X.replace([np.inf, -np.inf], np.nan).fillna(0)

artist_prior_cols = [
    "artist_song_count",
    "artist_avg_popularity",
    "artist_hit_rate",
]

lfm_feature_cols = [
    "lfm_matched",
    "lfm_listeners_log",
    "lfm_playcount_log",
    "lfm_play_per_listener_log",
]

metadata_cols = [
    "primary_artist", "lfm_artist_name", "lfm_api_status",
    "lfm_top_tag", "lfm_listeners", "lfm_playcount",
    "lfm_play_per_listener",
]

X_all_full = build_baseline_design_matrix(df)

# Baseline Random Forest models.
rf_b = RandomForestRegressor(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rfc_b = RandomForestClassifier(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1,
    class_weight="balanced_subsample"
)

rf_b.fit(X_train_base, y_train_reg)
rfc_b.fit(X_train_base, y_train_class)

# Content + NLP feature family: exclude Last.fm and artist-prior variables.
X_content_full = X_all_full.drop(
    columns=[c for c in artist_prior_cols + lfm_feature_cols + metadata_cols if c in
             X_all_full.columns],
    errors="ignore"
)
X_content_full = clean_numeric_matrix(X_content_full)
X_train_content = X_content_full.iloc[train_idx].copy()
X_test_content = X_content_full.iloc[test_idx].copy()

rf_content = RandomForestRegressor(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rfc_content = RandomForestClassifier(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1,
    class_weight="balanced_subsample"
)

```

```

)

rf_content.fit(X_train_content, y_train_reg)
rfc_content.fit(X_train_content, y_train_class)

content_reg_row = reg_metrics(y_test_reg, rf_content.predict(X_test_content))
content_reg_row["model"] = "Random Forest"
content_reg_row["version"] = "content_nlp"

content_prob = rfc_content.predict_proba(X_test_content)[: , 1]
content_pred = rfc_content.predict(X_test_content)
content_clf_row = clf_metrics(y_test_class, content_prob, content_pred)
content_clf_row["model"] = "Random Forest"
content_clf_row["version"] = "content_nlp"

# Content + NLP + Last.fm feature family: include Last.fm, exclude artist-prior.
X_lfm_full = X_all_full.drop(
    columns=[c for c in artist_prior_cols + metadata_cols if c in X_all_full.columns],
    errors="ignore"
)

X_lfm_full = clean_numeric_matrix(X_lfm_full)
X_train_lfm = X_lfm_full.iloc[train_idx].copy()
X_test_lfm = X_lfm_full.iloc[test_idx].copy()

rf_lfm = RandomForestRegressor(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

rfc_lfm = RandomForestClassifier(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1,
    class_weight="balanced_subsample"
)

rf_lfm.fit(X_train_lfm, y_train_reg)
rfc_lfm.fit(X_train_lfm, y_train_class)

lfm_reg_row = reg_metrics(y_test_reg, rf_lfm.predict(X_test_lfm))
lfm_reg_row["model"] = "Random Forest"
lfm_reg_row["version"] = "content_nlp_lastfm"

lfm_prob = rfc_lfm.predict_proba(X_test_lfm)[: , 1]
lfm_pred = rfc_lfm.predict(X_test_lfm)
lfm_clf_row = clf_metrics(y_test_class, lfm_prob, lfm_pred)
lfm_clf_row["model"] = "Random Forest"
lfm_clf_row["version"] = "content_nlp_lastfm"

# Supplementary artist-prior feature family.
X_up_full = X_all_full.drop(
    columns=[c for c in lfm_feature_cols + metadata_cols if c in X_all_full.columns],
    errors="ignore"
)

X_up_full = clean_numeric_matrix(X_up_full)
X_train_up = X_up_full.iloc[train_idx].copy()
X_test_up = X_up_full.iloc[test_idx].copy()

rf_u = RandomForestRegressor(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

rfc_u = RandomForestClassifier(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1,
    class_weight="balanced_subsample"
)

rf_u.fit(X_train_up, y_train_reg)
rfc_u.fit(X_train_up, y_train_class)

up_reg_row = reg_metrics(y_test_reg, rf_u.predict(X_test_up))
up_reg_row["model"] = "Random Forest"
up_reg_row["version"] = "artist_prior_supplement"

up_prob = rfc_u.predict_proba(X_test_up)[: , 1]
up_pred = rfc_u.predict(X_test_up)
up_clf_row = clf_metrics(y_test_class, up_prob, up_pred)
up_clf_row["model"] = "Random Forest"
up_clf_row["version"] = "artist_prior_supplement"

```

Listing 4. Top-Decile Lift Evaluation

```

def top_decile_lift(y_true_hit, scores, frac=0.1, n_mc=500, seed=RANDOM_STATE):
    """
    Evaluate how many true hits are captured in the top predicted fraction.
    This simulates the case where only the top 10% of tracks can be promoted.
    """
    n = len(y_true_hit)
    k = max(1, int(np.floor(frac * n)))

    order = np.argsort(-scores)
    top = order[:k]

    if hasattr(y_true_hit, "iloc"):
        hits_in_top = int(y_true_hit.iloc[top].sum())
    else:
        hits_in_top = int(y_true_hit[top].sum())

    total_hits = int(np.sum(y_true_hit))
    capture = hits_in_top / total_hits if total_hits else float("nan")
    precision_top = hits_in_top / k

    rng = np.random.RandomState(seed)
    random_captures = []

    for _ in range(n_mc):
        perm = rng.permutation(n)[:k]
        if hasattr(y_true_hit, "iloc"):
            random_hits = int(y_true_hit.iloc[perm].sum())

```

```

        else:
            random_hits = int(y_true_hit[perm].sum())
            random_captures.append(random_hits)

    random_capture_mean = (
        np.array(random_captures) / total_hits
    ).mean() if total_hits else float("nan")

    return {
        "n_test": n,
        "top_k": k,
        "hits_total": total_hits,
        "hits_in_top_k": hits_in_top,
        "hit_capture_rate": capture,
        "precision_in_top_k": precision_top,
        "random_capture_mean": float(random_capture_mean),
    }

y_hit_test = pd.Series(y_test_class).reset_index(drop=True)
biz_rows = []

model_scores = [
    ("baseline", "Baseline RF classifier", rfc_b.predict_proba(X_test_base)[: , 1]),
    ("content_nlp", "Content + NLP RF classifier", rfc_content.predict_proba(X_test_content)
     [: , 1]),
    ("content_nlp_lastfm", "Content + NLP + Last.fm RF classifier", rfc_lfm.predict_proba(
     X_test_lfm)[: , 1]),
    ("artist_prior_supplement", "Artist-prior RF classifier", rfc_u.predict_proba(X_test_up)
     [: , 1]),
]

for model_family, method, score in model_scores:
    row = top_decile_lift(y_hit_test, score)
    row["model_family"] = model_family
    row["method"] = method
    biz_rows.append(row)

biz_one_all = pd.DataFrame(biz_rows)
biz_one_all["lift_vs_random"] = (
    biz_one_all["hit_capture_rate"] / biz_one_all["random_capture_mean"]
)

family_order = {
    "baseline": 0,
    "content_nlp": 1,
    "content_nlp_lastfm": 2,
    "artist_prior_supplement": 3,
}

biz_one_all["family_order"] = biz_one_all["model_family"].map(family_order)
biz_one_all = (
    biz_one_all
    .sort_values(["family_order", "method"])
    .drop(columns=["family_order"])
    .reset_index(drop=True)
)

```

Listing 5. Feature Group Importance

```

engineered_project2_cols = {
    "energy_loudness",
    "danceability_energy",
    "valence_energy",
    "energy_minus_genre_mean",
    "danceability_minus_genre_mean",
    "valence_minus_genre_mean",
    "num_artists",
    "is_collab",
    "duration_min",
    "duration_log",
}

audio_baseline_cols = {
    "danceability", "energy", "loudness", "speechiness",
    "acousticness", "instrumentalness", "liveness", "valence",
    "tempo", "explicit",
}

artist_prior_cols = {
    "artist_song_count",
    "artist_avg_popularity",
    "artist_hit_rate",
}

def tag_feature_group(feature):
    if feature.startswith("title_"):
        return "NLP title features"
    if feature.startswith("lfm_"):
        return "Last.fm external artist features"
    if feature in artist_prior_cols:
        return "Internal artist-prior features"
    if feature in engineered_project2_cols:
        return "Engineered content features"
    if feature.startswith("track_genre") or feature.startswith("cluster") or feature.
        startswith("duration_bucket"):
        return "Genre / cluster / bucket features"
    if feature.startswith("PC"):
        return "PCA audio components"
    if feature in audio_baseline_cols:
        return "Original Spotify audio features"
    return "Other"

def summarize_group_importance(model, feature_names, model_label):
    importance_df = pd.DataFrame({
        "feature": feature_names,
        "importance": model.feature_importances_,
    })

    importance_df["group"] = importance_df["feature"].apply(tag_feature_group)

```

```

group_df = (
    importance_df
    .groupby("group")["importance"]
    .sum()
    .sort_values(ascending=False)
    .reset_index()
)

group_df["model"] = model_label
return group_df

group_importance_tables = [
    summarize_group_importance(
        rfc_content,
        X_train_content.columns.tolist(),
        "Content + NLP RF classifier"
    ),
    summarize_group_importance(
        rfc_lfm,
        X_train_lfm.columns.tolist(),
        "Content + NLP + Last.fm RF classifier"
    ),
    summarize_group_importance(
        rfc_u,
        X_train_up.columns.tolist(),
        "Artist-prior RF classifier"
    ),
]

group_importance_summary = pd.concat(
    group_importance_tables,
    ignore_index=True
)

```

```

X_test=Xte,
y_train_reg=y_train_reg,
y_test_reg=y_test_reg,
y_train_class=y_train_class,
y_test_class=y_test_class,
version_name=version_name,
label_name=label_name,
)

if reg_row is not None:
    xgb_reg_rows.append(reg_row)
    xgb_clf_rows.append(clf_row)

xgb_reg_df = pd.DataFrame(xgb_reg_rows)
xgb_clf_df = pd.DataFrame(xgb_clf_rows)

```

Listing 6. XGBoost Robustness Check

```

def train_xgboost_family(
    X_train,
    X_test,
    y_train_reg,
    y_test_reg,
    y_train_class,
    y_test_class,
    version_name,
    label_name,
):
    """
    Train one XGBoost regressor and one XGBoost classifier for a feature family.
    """
    if not XGBOOST_AVAILABLE:
        return None, None, None, None

    xgb_reg = XGBRegressor(
        n_estimators=200,
        max_depth=5,
        learning_rate=0.05,
        subsample=0.9,
        colsample_bytree=0.9,
        objective="reg:squarederror",
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )

    pos_weight = (len(y_train_class) - np.sum(y_train_class)) / np.sum(y_train_class)

    xgb_clf = XGBClassifier(
        n_estimators=200,
        max_depth=5,
        learning_rate=0.05,
        subsample=0.9,
        colsample_bytree=0.9,
        objective="binary:logistic",
        eval_metric="logloss",
        scale_pos_weight=pos_weight,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )

    xgb_reg.fit(X_train, y_train_reg)
    xgb_clf.fit(X_train, y_train_class)

    reg_row = reg_metrics(y_test_reg, xgb_reg.predict(X_test))
    reg_row["model"] = "XGBoost"
    reg_row["version"] = version_name
    reg_row["n_features"] = X_test.shape[1]

    prob = xgb_clf.predict_proba(X_test)[:, 1]
    pred = xgb_clf.predict(X_test)

    clf_row = clf_metrics(y_test_class, prob, pred)
    clf_row["model"] = "XGBoost"
    clf_row["version"] = version_name
    clf_row["n_features"] = X_test.shape[1]

    return xgb_reg, xgb_clf, reg_row, clf_row

xgb_reg_rows = []
xgb_clf_rows = []

xgb_jobs = [
    (X_train_content, X_test_content, "content_nlp", "Content + NLP"),
    (X_train_lfm, X_test_lfm, "content_nlp_lastfm", "Content + NLP + Last.fm"),
    (X_train_up, X_test_up, "artist_prior_supplement", "Content + NLP + internal artist-prior"),
]

for Xtr, Xte, version_name, label_name in xgb_jobs:
    xgb_reg, xgb_clf, reg_row, clf_row = train_xgboost_family(
        X_train=Xtr,

```